# NLU course project

*Seyed Morteza Mojtabavi (256328)*

University of Trento

seyed.mojtabavi@studenti.unitn.it

The following is the report for the first part of the project: Language Modeling.

## 1. Introduction

This part of the project builds an autoregressive next-word predictor and improves it step by step, using test-set Perplexity (PPL, lower is better) as the evaluation metric throughout. Part A starts from a vanilla Elman RNN and incrementally swaps in an LSTM cell, adds embedding/output dropout, and replaces SGD with AdamW, followed by a sequential hyperparameter search. Part B instead restarts from Part A's vanilla RNN baseline and layers in three regularization techniques from Merity et al. (2017) — weight tying, variational dropout, and non-monotonically triggered ASGD — again followed by targeted tuning, with the requirement that the final Test PPL beat Part A's best LSTM result. Every modification was applied one at a time and validated before moving to the next. All training ran on Google Colab rather than a dedicated GPU machine, which limited how much I could sanity-check the code beforehand and how exhaustive the hyperparameter search could be.

## 2. Implementation details

### 2.1. Part A

Starting from a vanilla `nn.RNN` baseline [1], I swapped it for `nn.LSTM` [2] with no other change, to isolate the architectural effect. I then added two `nn.Dropout` [3] layers — one right after the embedding lookup, one right before the output projection — as standalone modules rather than relying on `nn.LSTM`'s built-in dropout argument, which has no effect at `num_layers=1`. Finally, I replaced SGD with AdamW [4] while keeping everything else fixed, isolating the optimizer's own contribution. Hyperparameters (learning rate, hidden/embedding size, gradient clipping) were then tuned sequentially — fixing each parameter at its best value before searching the next, rather than a full grid — to keep the search tractable. Batch size and patience were left at their defaults throughout.

### 2.2. Part B

Part B reuses Part A's vanilla RNN as its starting point, not the LSTM — the target this part must beat is Part A's best LSTM PPL, not its own architecture. On top of the RNN, I implemented weight tying by directly assigning the output projection's weight matrix to the embedding matrix, which requires `emb_size == hidden_size` (enforced with an explicit check) [5]. I then implemented variational dropout myself as a small custom module, since PyTorch has no built-in layer that samples one dropout mask per sequence and reuses it at every timestep instead of resampling per step [6]. Finally, I implemented NT-AvSGD [7] following the trigger rule in Merity et al. (2017) [8]: validation loss is monitored every epoch, and the optimizer switches from SGD to ASGD once it fails to beat its best value from more than `non_mono` epochs ago. While ASGD is active, the model's live weights are temporarily swapped for ASGD's running average before every evaluation, then restored, so validation and testing always reflect the averaged parameters.

## 3. Results

All figures are test-set Perplexity (PPL), computed as the exponential of the average cross-entropy loss over all non-padding tokens in the PTB test split; validation PPL (same formula, dev split) drove early stopping and the hyperparameter search.

### 3.1. Part A

Moving from RNN to LSTM gave the largest single architectural gain, consistent with LSTM gating mitigating vanishing gradients. The two dropout layers added a further, smaller improvement. Switching SGD for AdamW produced the single biggest reduction in this part, reflecting AdamW's per-parameter adaptive rates and decoupled weight decay. The subsequent sequential search — mainly enlarging hidden/embedding size to 400 and tightening gradient clipping to 0.1 — reached a final Test PPL of **115.73**, well under the required 250.

Table 1: *Best perplexity results for each configuration (Part A).*

| Configuration | Val PPL | Test PPL |
|---|---|---|
| Baseline RNN | 169.33 | 161.83 |
| Baseline LSTM | 154.68 | 149.52 |
| LSTM + Dropout | 151.66 | 146.90 |
| + AdamW | 133.25 | 122.84 |
| + Tuning: batch_size=16 | 132.97 | 122.48 |
| + Tuning: hidden_size=400 | 125.86 | 117.88 |
| + Tuning: lr=0.001 (unchanged) | 125.86 | 117.88 |
| + Tuning: emb_size=400 | 125.29 | 116.66 |
| **Final (clip=0.1)** | **123.43** | **115.73** |

### 3.2. Part B

A first tuning pass over the learning rate was necessary since the default SGD rate diverged entirely; `lr=5` stabilized training at a Test PPL on par with Part A's LSTM baseline (149.52), despite this RNN having no gating mechanism. Weight tying alone barely changed the score, but it made embedding size a meaningful hyperparameter, and tuning it gave the first result below that baseline. Variational dropout then delivered the largest single gain, and NT-AvSGD with a tight trigger window (`non_mono=1`, since wider windows 2–5 triggered too late) gave the final improvement. The final model reaches Test PPL **128.73**, below both the 250 threshold and Part A's LSTM baseline.

Table 2: *Best perplexity results for each configuration (Part B).*

| **Configuration** | **Val PPL** | **Test PPL** |
|---|---|---|
| Baseline RNN + SGD (lr=1) | 7891.02 | 7918.07 |
| + Tuning: lr=5 | 156.76 | 150.92 |
| + Weight Tying (emb=hidden=400) | 155.42 | 150.84 |
| + Tuning: emb_size=200 | 149.24 | 145.35 |
| + Variational Dropout | 140.89 | 136.01 |
| **+ NT-AvSGD (non_mono=1)** | **132.97** | **128.73** |

# 4. References


[1] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur, "Recurrent neural network based language model," in *INTERSPEECH 2010*, 2010, pp. 1045–1048.

[2] PyTorch, "Long short-term memory (lstm)," https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html.

[3] ——, "Dropout," https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout.html.

[4] ——, "Adamw," https://docs.pytorch.org/docs/stable/generated/torch.nn.AdamW.html.

[5] O. Press and L. Wolf, "Using the output embedding to improve language models," *arXiv preprint arXiv:1608.05859*, 2017.

[6] Y. Gal, "A theoretically grounded application of dropout in recurrent neural networks," *arXiv preprint arXiv:1512.05287*, 2016.

[7] PyTorch, "Asgd," https://docs.pytorch.org/docs/stable/generated/torch.optim.ASGD.html.

[8] S. Merity, N. S. Keskar, and R. Socher, "Regularizing and optimizing lstm language models," in *International Conference on Learning Representations (ICLR)*, 2018, code available at https://github.com/salesforce/awd-lstm-lm.